PROJECT DOCUMENTATION

Subject: Networking Issues in Mobile Systems
Part 1 – Conceptual Design
Project topic: Mobile Currency Exchange System

1. General Information

Project name	NBP Currency Exchange – a network-resilient mobile currency exchange application
Project authors	Efe Saka (student ID 55934) – individual project
Field of study	Computer Science
Year / semester	2026/2027, 7th semester
Supervisor	Marcin Kacprowicz
Submission date	01.11.2026
Repository	https://github.com/EfeSaka02/currency-exchange, demonstrated revision: tag `m1-design`

2. Project Description

2.1. Project Objective

The aim of the project is to create a mobile application that enables virtual currency exchange. Users can browse current exchange rates published by the National Bank of Poland (NBP) and perform simulated buy and sell transactions using a virtual PLN balance. The system is designed to remain reliable under unstable network conditions: it handles connection loss, timeouts and repeated requests without losing data and without executing a transaction twice. The user value is a safe environment in which to learn how currency exchange works, with clear and predictable application behaviour even when the connection is poor.

2.2. Project Scope

The system consists of three modules and one external data source:

Module	Role
Mobile application	User interface. Shows exchange rates, the virtual wallet and the transaction history, and lets the user submit buy and sell orders. Stores the last downloaded rates and unconfirmed orders locally, so that rates can be displayed offline and no order is lost.
Web Service (backend)	REST API between the application and the data sources. Authenticates users, fetches and caches rates from NBP, validates and executes transactions atomically, and guarantees that a repeated request is processed only once.
Database	Stores users, wallet balances, exchange rates and transactions. Enforces integrity with keys and constraints.
NBP Web API (external)	Official source of bid/ask exchange rates (table C) and rate history. Accessed only by the backend.

Out of scope: real money, real payments and bank integrations. All balances are virtual; each new user starts with a virtual balance of 10,000 PLN.

Pricing rule: a purchase of a foreign currency is calculated at the NBP *ask* rate and a sale at the NBP *bid* rate (NBP table C). The amount in PLN is rounded to two decimal places.

3. System Requirements

3.1. Functional Requirements

ID	Function	Description
FR-01	Registration and login	A user can create an account (e-mail and password) and log in. A new account receives a virtual balance of 10,000 PLN.
FR-02	Current rates	The application shows the latest bid and ask rates of all currencies available in NBP table C, with the rate date.
FR-03	Rate history	The user can view the rate history of one currency for the last 30 days.
FR-04	Wallet	The user can view the balance in PLN and in each foreign currency.
FR-05	Buy currency	The user buys a chosen amount of a foreign currency for PLN at the current ask rate. The order is rejected if the PLN balance is insufficient.
FR-06	Sell currency	The user sells a chosen amount of a foreign currency for PLN at the current bid rate. The order is rejected if the currency balance is insufficient.
FR-07	Transaction history	The user can list past transactions with type, amount, rate, status and time.
FR-08	Offline rates	When there is no connection, the application shows the last downloaded rates together with their download time and an offline indicator.
FR-09	Idempotent orders	Every order carries a unique request key generated by the application. The backend executes an order with a given key at most once and returns the original result for repeated requests.
FR-10	Automatic retry	An order whose outcome is unknown (timeout, dropped connection) is stored locally as UNCONFIRMED and resent with the same key, with increasing delays, until a definite answer is received.
FR-11	Order status check	The application can ask the backend for the status of an order by its request key, e.g. after reconnecting.
FR-12	Rate synchronization	The backend periodically downloads rates from NBP and stores them. If NBP is unavailable, the last stored rates are served and marked as stale.
FR-13	Network feedback	The application shows the connection state and clear messages for failures (offline, timeout, order unconfirmed, rate changed).

3.2. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Consistency	A transaction is atomic: the transaction record and both balance changes are saved together or not at all (database transaction).
NFR-02	Reliability	No order is executed twice and no confirmed order is lost, regardless of retries, dropped connections or application restarts.
NFR-03	Performance	The rates screen shows cached data immediately (under 0.5 s) and refreshes in the background; a network request times out after 10 s.
NFR-04	Offline tolerance	Browsing rates and the wallet from cached data works without a connection; buying and selling requires a connection.
NFR-05	Security	All communication uses HTTPS (TLS); passwords are stored only as salted hashes; API calls are authorized with a token.
NFR-06	Data accuracy	Money and rates use fixed-point decimal types (NUMERIC), never floating point.
NFR-07	Usability	Every failure is explained to the user in plain language, with the possible next action.
NFR-08	Maintainability	Separate layers (UI, repository, API client; controller, service, data access) so each part can be tested independently.
NFR-09	Testability	Network failures can be simulated (delay, drop, offline) to test every scenario in section 6.3.

4. UML Diagrams

4.1. Use Case Diagram

The user is the only human actor. The NBP API is an external actor that provides the data for rate synchronization. The use cases *Show cached rates* and *Retry unconfirmed order* are extensions that are triggered only by network problems.

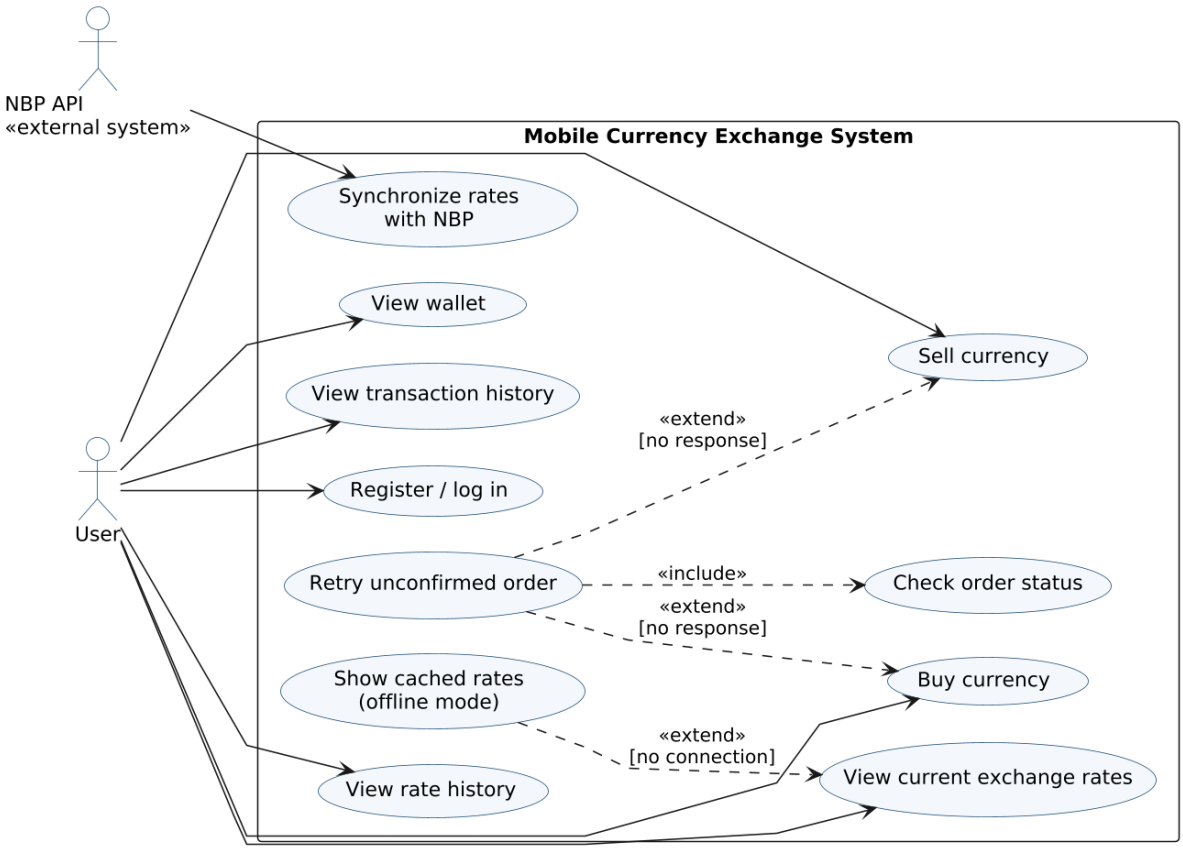


Figure 1. Use case diagram

4.2. Class Diagram

Figure 2 shows the domain model: users hold balances per currency, and every transaction refers to a user, a currency and the exchange rate applied. The *requestKey* attribute of *Transaction* is what makes repeated orders safe.

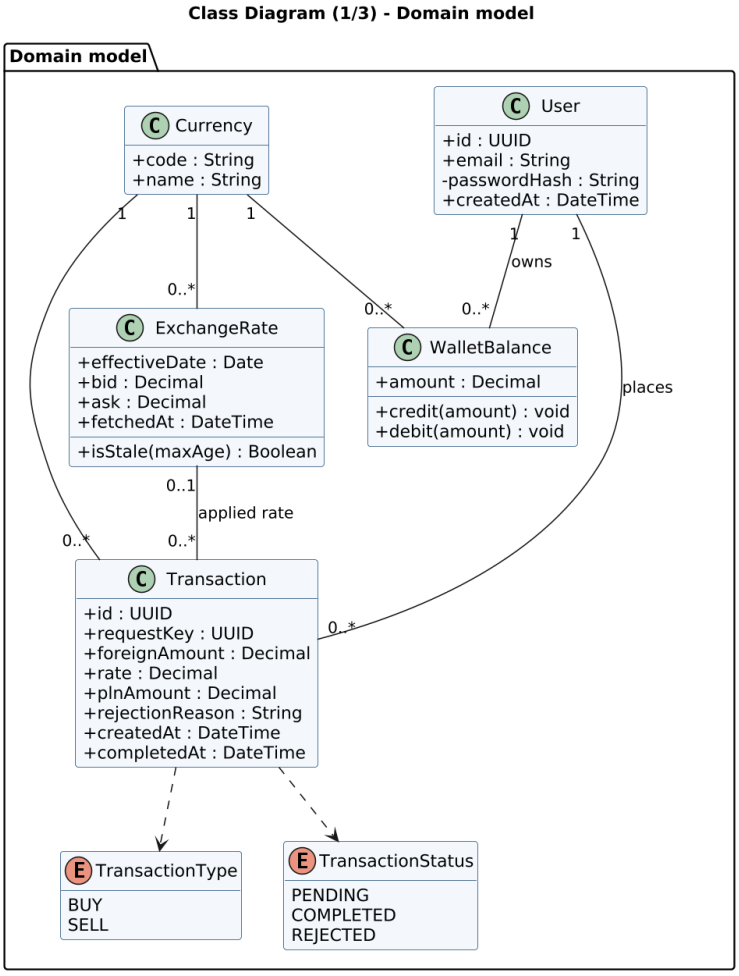


Figure 2. Class diagram (1/3): domain model

Figure 3 shows the backend service classes: *OrderService* executes orders idempotently, *RateService* serves and synchronizes rates, and *NbpClient* is the only class that talks to NBP. Figure 4 shows the classes of the mobile application that handle caching and retries.

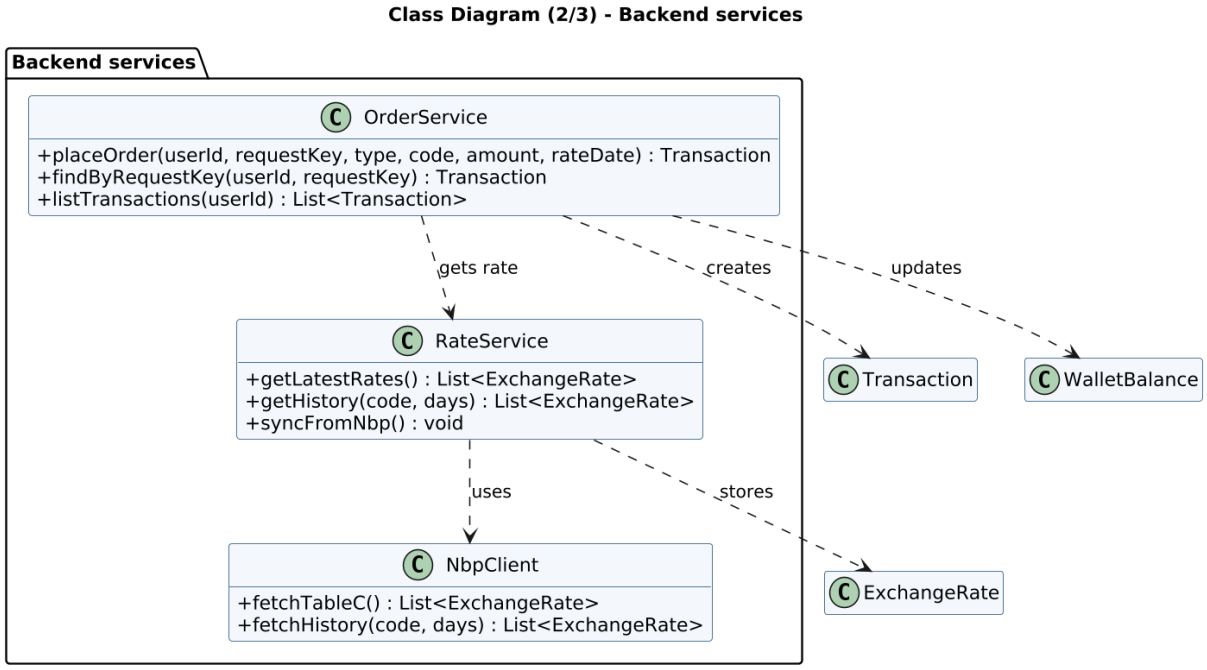


Figure 3. Class diagram (2/3): backend services

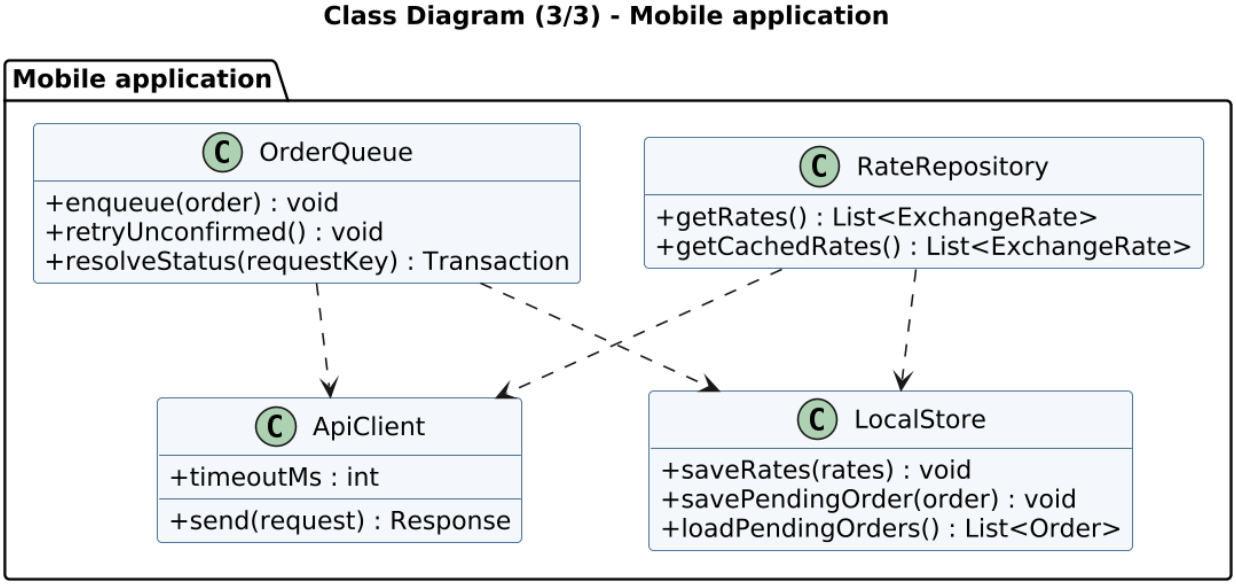


Figure 4. Class diagram (3/3): mobile application

5. Database Design

The relational database has five tables. Primary keys are marked PK and foreign keys FK. Money and rates use NUMERIC types.

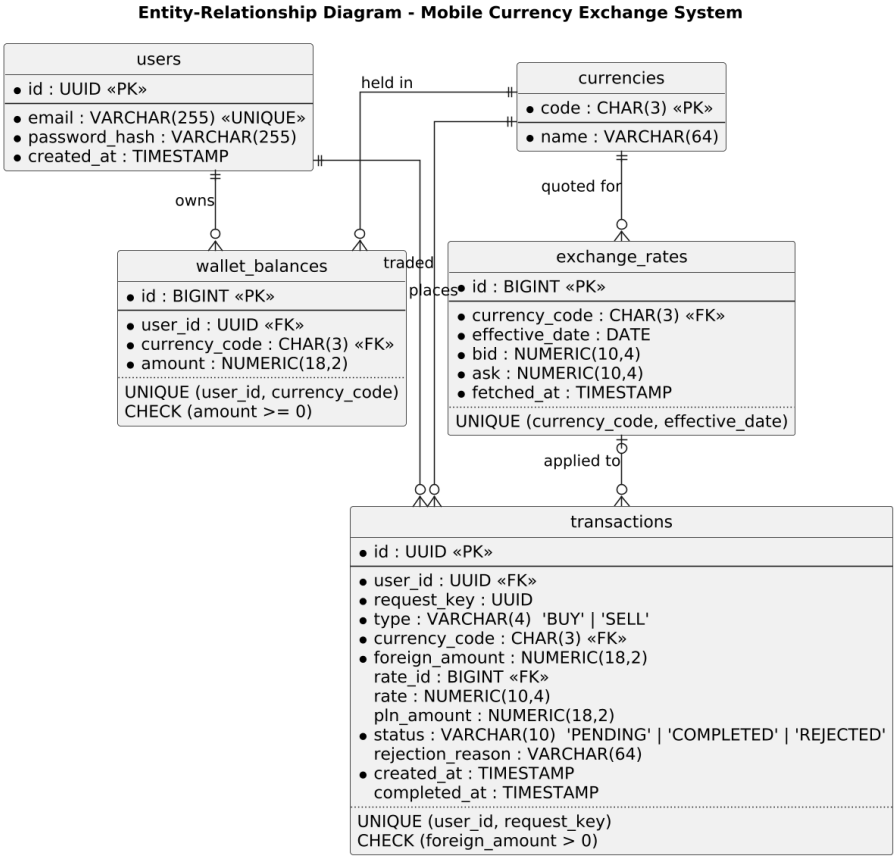


Figure 5. Entity-Relationship Diagram

Table	Purpose and key columns	Relationships and constraints
users	Registered users. **id** (UUID, PK), email, password_hash, created_at.	email is UNIQUE.
currencies	Supported currencies. **code** (CHAR(3), PK), name.	Referenced by balances, rates and transactions.
wallet_balances	Virtual balance of a user in one currency (PLN is a currency too). **id** (PK), user_id (FK), currency_code (FK), amount NUMERIC(18,2).	UNIQUE (user_id, currency_code); CHECK (amount >= 0), which prevents a negative balance even under concurrent requests.
exchange_rates	Rates downloaded from NBP table C. **id** (PK), currency_code (FK), effective_date, bid and ask NUMERIC(10,4), fetched_at.	UNIQUE (currency_code, effective_date), so a re-synchronization never creates duplicates.
transactions	Every buy/sell order. **id** (UUID, PK), user_id (FK), request_key (UUID), type, currency_code (FK), foreign_amount, rate_id (FK), rate, pln_amount, status, rejection_reason, created_at, completed_at.	**UNIQUE (user_id, request_key)** is the database-level guard against duplicate execution. CHECK (foreign_amount > 0). status: PENDING, COMPLETED, REJECTED.

On the mobile device a small local store (SQLite) keeps two things: the last downloaded rates with their download time, and the list of orders that have been submitted but not yet confirmed (request key, order data, number of attempts). It is a cache and a retry buffer, not a second source of truth: the server database always decides.

6. System Architecture

6.1. Modules and Data Flow

The mobile application never talks to NBP directly. It talks only to the backend over HTTPS with JSON. The backend downloads rates from NBP on a schedule, stores them in the database and serves them to all clients. This keeps the application independent of NBP availability and reduces mobile data usage.

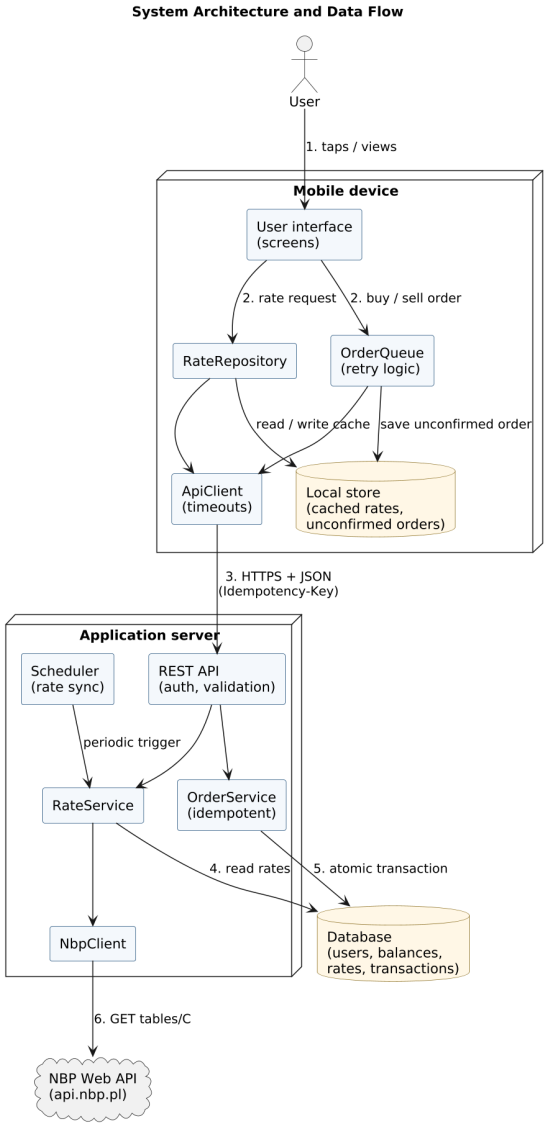


Figure 6. Architecture and logical data flow (numbers show the order of steps)

Data flow in short: (1) the user acts in the UI; (2) the repository or the order queue prepares the request; (3) the API client sends it over HTTPS; (4) rate requests are answered from the database; (5) orders are executed in one database transaction; (6) the scheduler refreshes rates from NBP in the background.

Proposed technology (can be adjusted without changing the design): React Native for the mobile application, a Node.js REST backend, PostgreSQL as the database and SQLite on the device.

6.2. API Outline

Base path: `/api/v1`. All endpoints except registration and login require an authorization token. Order creation requires the header `Idempotency-Key` containing the request key.

Method and path	Purpose	Main responses
POST /auth/register	Create an account (FR-01).	201; 409 e-mail exists
POST /auth/login	Log in, receive a token (FR-01).	200; 401 wrong credentials
GET /rates	Latest bid/ask rates with rateDate, fetchedAt and a stale flag (FR-02).	200; 503 no rates stored yet
GET /rates/{code}/history?days=30	Rate history of one currency (FR-03).	200; 404 unknown currency
GET /wallet	Balances of the user (FR-04).	200
POST /orders	Place a buy or sell order. Body: type, currencyCode, amount, rateDate (FR-05, FR-06, FR-09).	201 executed; **200 duplicate key (original result)**; 409 RATE_CHANGED; 422 INSUFFICIENT_FUNDS; 503 RATES_UNAVAILABLE
GET /orders	Transaction history with paging (FR-07).	200
GET /orders/by-key/{requestKey}	Status of an order after an unknown outcome (FR-11).	200 with status; 404 order never reached the server

Example order request:

```
POST /api/v1/orders
Idempotency-Key: 7b1c2f0e-5a3d-4c1a-9e55-0c1d2e3f4a5b
{ "type": "BUY", "currencyCode": "USD", "amount": "100.00", "rateDate": "2026-10-30" }
```

6.3. Network-Failure Scenarios

The central design rule is: **an order is identified by a request key created on the device before the first send, and the same key is reused for every retry.** If the outcome of a request is unknown, the application never creates a new order; it asks again with the same key. The server executes a key at most once and answers repeats with the stored result.

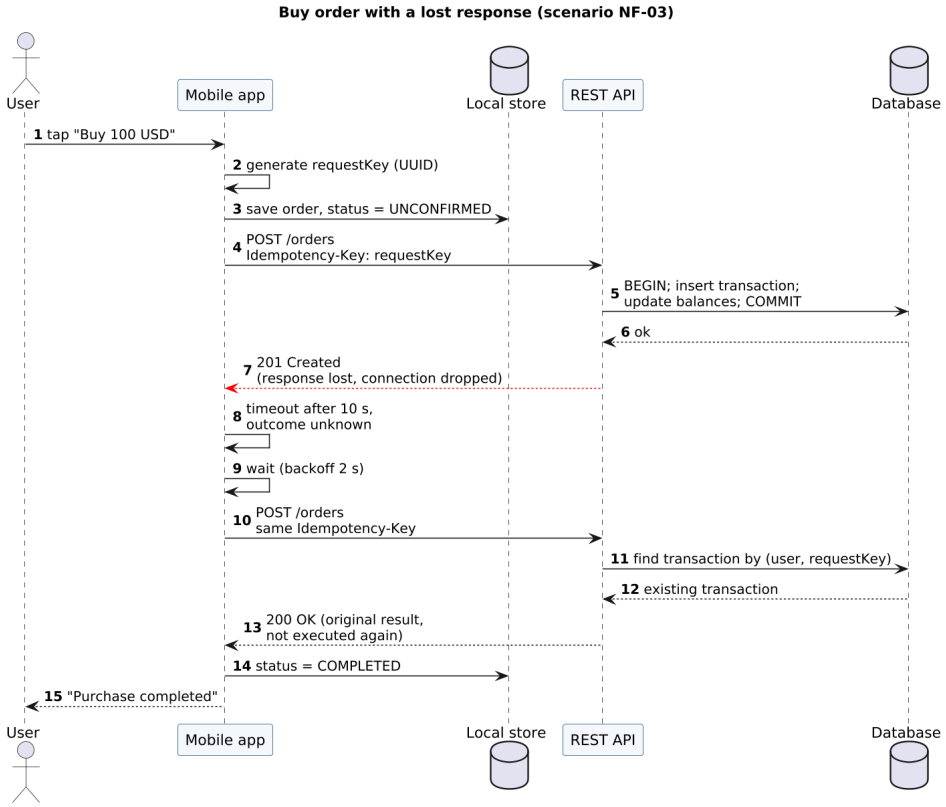


Figure 7. Sequence diagram of scenario NF-03 (lost response)

ID	Situation	How it is detected	System behaviour and what the user sees
NF-01	No connection when the rates screen opens	Connectivity check, request fails immediately	Cached rates are shown with their download time and an "offline" banner. Buy and Sell are disabled, because an order needs a current rate.
NF-02	Slow network, rate request exceeds 10 s	Client timeout	Cached rates stay visible, a refresh indicator is shown, the request is retried in the background. No error dialog is needed.
NF-03	Connection drops after the order was sent, response lost (outcome unknown)	Timeout or connection reset after sending	The order stays UNCONFIRMED in the local store. The app retries with the **same key** after 2, 4, 8, ... s. The server finds the key and returns the original result. User sees "Processing", then the final result. No double purchase.
NF-04	Request never reached the server	Timeout; later, status check returns 404	Retry with the same key executes the order for the first time. Nothing is lost.
NF-05	User taps Buy twice or re-submits manually	Same request key for the same pending order	The second request is answered with the existing transaction (HTTP 200). One transaction only; the button is disabled while an order is in progress.
NF-06	Switch between Wi-Fi and mobile data, or temporary loss of signal in a tunnel	Connection reset or timeout	Handled as NF-03. When connectivity returns, the app resolves all UNCONFIRMED orders first, then refreshes rates and the wallet.
NF-07	Application is closed or the phone restarts while an order is unconfirmed	On start-up the local store contains UNCONFIRMED orders	The app resumes retrying; if needed it queries GET `/orders/by-key`. The user sees the final status in the history.

ID	Situation	How it is detected	System behaviour and what the user sees
NF-08	Server error (HTTP 5xx) or backend temporarily down	5xx status or no answer	Treated as unknown outcome: retry with the same key. After the maximum number of attempts the order is shown as "Unconfirmed - check later" and is never silently dropped.
NF-09	Corrupted or incomplete response	JSON cannot be parsed	Treated as unknown outcome (NF-03); the status check by key gives the true result.
NF-10	NBP API unavailable for the backend	Rate synchronization fails	The backend serves the last stored rates with stale = true. The app shows the rate date. If stored rates are older than a configurable limit (e.g. 4 days, covering weekends and holidays), orders are rejected with RATES_UNAVAILABLE.
NF-11	A new NBP table is published between showing the rate and sending the order	rateDate in the order differs from the current rate date	The server answers 409 RATE_CHANGED with the new rate. The app shows the new rate and asks the user to confirm. The order is not executed at an unseen price.

Retry parameters (initial values): request timeout 10 s; delays 2, 4, 8, 16 s with a small random jitter; at most 6 automatic attempts per session; unconfirmed orders are retried again at the next application start and when connectivity returns.

7. Project Implementation Plan

The project is carried out by one author, who is responsible for all parts: analysis and design, mobile application, backend, database, testing and documentation. The work is divided into three phases that follow the course milestones.

Phase	Scope	Result
1. Analysis and design (M1)	Requirements, UML diagrams, database design, architecture, API outline, network-failure scenarios.	This document, editable diagram sources, repository with the design revision.
2. Working client/server prototype (M2)	Backend with NBP synchronization and database; mobile screens for rates, wallet, history; buy and sell orders over the API.	Application that works on a stable connection.
3. Network-resilient application (M3)	Local cache, offline mode, request keys, retry queue, status check, user messages; tests that simulate delays, drops and offline mode (scenarios NF-01 to NF-11).	Application that behaves correctly under network failures, with test evidence.

Main risks: change of the NBP API (mitigation: all NBP access is isolated in *NbpClient*); scope growth (mitigation: virtual money only, fixed list of functions in section 3.1); time (mitigation: network resilience is implemented only for orders and rates, the most critical flows).

8. Conclusions and Possible Extensions

The design describes a currency exchange application whose main quality is correct behaviour on an unreliable mobile network. The key decisions are: a backend between the application and NBP, a request key for every order, atomic database transactions with a uniqueness constraint, and a local cache with a retry buffer on the device.

Possible extensions after the project is completed:

- Rate alerts and push notifications when a currency reaches a chosen value.
- Rate history charts with longer periods and comparison of currencies.

- Support for NBP table A and B currencies and a currency converter.
- Limit orders: execute an order automatically when the rate reaches a target.
- Profit and loss statistics for the user's virtual portfolio.
- Server-side synchronization via push messages instead of periodic polling.
- Biometric login and multiple devices per user.

9. Sources

- National Bank of Poland, NBP Web API documentation, https://api.nbp.pl/en.html (exchange rate tables A, B, C and rate series).
- Course materials of Networking Issues in Mobile Systems (NIMB2026): lectures, lecture slides, laboratory project handbook and report template.
- Fielding, R., Nottingham, M., Reschke, J. (2022). RFC 9110: HTTP Semantics, IETF – idempotent methods and status codes.
- IETF HTTP API Working Group, The Idempotency-Key HTTP Header Field (Internet-Draft).
- Stripe documentation, Idempotent requests, https://docs.stripe.com/api/idempotent_requests – practical example of the request-key pattern.
- PostgreSQL documentation, Numeric types and Constraints, https://www.postgresql.org/docs/.
- PlantUML documentation, https://plantuml.com/ – tool used for the editable diagram sources.
- React Native documentation, https://reactnative.dev/docs/network – networking in the mobile application.

Appendix A. Submitted Files and Repository

File	Content
M1_Project_Documentation_Currency_Exchange.pdf	This report.
01_use_case.puml	Editable source of Figure 1 (PlantUML).
02a_class_domain.puml, 02b_class_backend.puml, 02c_class_mobile.puml	Editable sources of Figures 2, 3 and 4.
03_erd.puml	Editable source of Figure 5.
04_architecture.puml	Editable source of Figure 6.
05_sequence_buy_retry.puml	Editable source of Figure 7.

Git repository: https://github.com/EfeSaka02/currency-exchange
Demonstrated revision: tag `m1-design` (contains this report and all diagram sources)